

项目申请书

项目名称: VSAG中Merge能力支持和性能优化

申请人: 蓝浩

邮箱: 1816391428@qq.com

日期: 2026.8.13

1. 项目背景

仓库地址: <https://github.com/antgroup/vsag>

随着 RAG、推荐系统、图片和文本检索等应用的发展, 向量数据库需要持续写入大量高维向量, 并经常产生多个独立的图索引。当小索引持续积累、数据分片重组或冷热数据需要统一管理时, 需要将多个已有索引合并为一个新的全局索引。

目前最直接的方法是从全部原始向量重新构建索引, 但会产生大量重复的距离计算和图搜索开销。已有子索引已经保存了部分有效的近邻关系和导航结构, 因此本项目希望研究如何充分复用这些结构, 以低于 Full Rebuild 的成本完成图索引合并, 同时保持较好的 Recall、QPS 和连通性。

2. 前期调研

我已经对图索引 Merge 方向进行了初步调研, 目前认为其核心问题主要包括:

- 已有结构复用:** 判断已有子图中的哪些邻居关系在全局图中仍然有效;
- 跨图结构建立:** 以较低的距离计算开销发现不同子图之间的近邻和导航关系;
- 全局结构修复:** 保证合并后的图具有较好的可达性、连通性和搜索质量。

此外还需要考虑多索引 Merge、内存与 IO、并行化以及大小索引非对称合并等工程问题。

前期主要调研了 FGIM、HNSW-Merger 和 MIM 等工作。FGIM 通过 Cross Querying 和 Refinement 构造跨图邻居; HNSW-Merger 利用已有邻居和反向邻居关系减少跨图搜索, 并针对非对称 Merge 和流式处理进行了优化; MIM 则利用 Pivot、Follower 和 Merge-order Graph 减少多索引合并中的重复搜索。

3. 拟研究方法

本项目计划重点探索:

基于节点重要性的非均匀 Merge Budget 分配。

现有 Merge 方法通常会对大量节点执行类似规模的跨图搜索或候选扩展, 但不同节点对于图搜索的重要性可能存在明显差异。

初步考虑使用节点的反向邻居数量 (入度) 作为低成本的重要性指标:

$$d_{in}(v) = |u \mid (u, v) \in E|$$

入度可以直接从已有图索引中统计, 无需额外执行向量搜索。

首先通过实验分析节点入度与以下指标之间的关系:

- 节点被访问/展开次数;
- Merge 前后的邻居变化程度;
- 跨图边数量;
- 查询过程中的搜索重要性。

如果能够观察到稳定相关性，则进一步将固定的 Merge Budget B

由传统的均匀分配： $b_i = B/N$

改为： $b_i = f(d_{in}(v_i))$

使重要节点获得更多 Cross Search / Candidate Budget，而普通节点使用更少预算。

实验将重点比较 **Uniform Budget** 与 **Indegree-aware Budget** 在相同计算量下的 Recall-QPS，以及在相同 Recall 下的距离计算次数和 Merge Time。

4. VSAG 实现与实验

计划选择 VSAG 中至少一种图索引实现 Merge 能力，包括：

- 读取并复用已有邻接结构；
- 建立跨图连接；
- 支持多个已有索引输入；
- 提供必要配置项；
- 增加单元测试和异常处理；
- 保证 Merge 后索引能够正常搜索和序列化。

Benchmark 至少在两个数据集或数据规模上进行，并测试不同：

- 子索引数量；
- 数据划分方式；
- 索引规模；
- Merge Budget。

主要统计：

- Merge / Rebuild Time；
- Distance Computation；
- Peak Memory；
- Recall；
- QPS / Latency；
- 节点可达性和连通性。

在算法功能稳定后，使用 perf、perf stat、火焰图等工具分析 CPU Hotspot、Cache Miss、内存带宽和锁竞争，并进行针对性的 C++ 性能优化。

5. 项目规划

第 1 ~ 2 周：Baseline 与节点重要性分析

- 熟悉 VSAG 相关图索引实现；
- 搭建 Merge Benchmark；
- 实现 Full Rebuild 等 Baseline；
- 统计入度、访问次数、展开次数等节点指标；
- 验证入度作为节点重要性指标的可行性。

第 3 ~ 4 周：Merge 算法原型

- 实现基础 Cross-Graph Merge;
- 实现 Uniform Budget;
- 实现基于入度的非均匀 Budget;
- 对比 Recall、距离计算次数和 Merge Time。

第 5 ~ 6 周：VSAG 集成与优化

- 完成 Merge API 和配置项;
- 支持多个索引;
- 增加单元测试;
- 使用 perf 等工具进行性能 Profiling 和优化。

第 7 ~ 8 周：完整实验与 PR

- 在不同数据集、分片数量和规模下完成 Benchmark;
- 整理原始实验数据;
- 完成设计文档和性能分析报告;
- 向 VSAG 提交可评审 PR，并根据 Review 修改。

6. 预期成果

1. 在 VSAG 中实现至少一种图索引的 Merge 能力，并提交可评审 PR;
2. 提供配置项、单元测试、Benchmark 脚本和使用文档;
3. 完成基于节点入度的非均匀 Merge Budget 方法实验;
4. 在至少两个数据集或规模配置上与 Full Rebuild 进行比较;
5. 力争在至少一个配置下获得不低于 1.5 倍的 Merge / Build 加速，或显著减少距离计算，同时将 Recall 下降控制在 1 个百分点以内;
6. 输出完整的实验与性能 Profiling 报告。